

Curbside

Final Project Design Document

A neighborhood giveaway platform for New York City

Team: Shohruzбек Abdumuminov, Sanjar Melikov, Ilobi Michael

1. Background and Problem

In New York, usable items are left on the curb every day: furniture, electronics, books, and other household goods. Some are picked up by passersby, but most sit untouched until it rains or they are thrown out. The core problem is timing and visibility, because the person giving an item away and the person who would actually want it have no way to find each other before it is gone. Curbside closes that gap. A giver posts an item in a few seconds with a photo, a short description, and its location. The post appears on a live map and feed for nearby users, and anyone interested can reserve the item for a limited window so no one makes a wasted trip. Posts expire automatically, so the feed stays current. The result is less neighborhood waste and a fast, easy way to find local giveaways.

2. System Architecture

Curbside has three layers: a React client that runs in the browser, a stateless Express REST API, and a MongoDB database. The client renders the map, feed, and post forms and talks to the API over HTTPS, attaching a JWT on requests that need authentication. The API validates input, enforces ownership, and handles all reads and writes. MongoDB stores users and items, with a 2dsphere index that powers queries for nearby items and a TTL index that expires posts automatically. Item photos are uploaded from the client to Cloudinary, which returns a hosted URL stored on the item, so image data never sits in the API or the database. Both the client and the API are containerized with Docker and published to Docker Hub.

Request flow: the React client sends requests to the Express API, which checks the JWT, validates the input, and reads from or writes to MongoDB. Image uploads go straight from the client to Cloudinary, and the returned URL is saved on the item record.

3. Technology Stack

- Frontend: React, Tailwind CSS, React Router, and Leaflet for interactive maps.
- Backend: Node.js and Express, with JSON Web Tokens for authentication, bcrypt for password hashing, and Mongoose as the data layer.
- Database: MongoDB, using a 2dsphere geospatial index and a TTL index for expiration.
- Image hosting: Cloudinary, with uploads handled in the browser and referenced by URL.
- Tooling and deployment: Git and GitHub for version control and issue tracking; Docker and Docker Hub for containerization and distribution.

4. Data Models

- User: id, username, email, passwordHash, createdAt.
- Item: id, title, description, photoUrl, category, location (GeoJSON Point as [lng, lat]), status (available / reserved / claimed), postedBy, reservedBy, reservedUntil, createdAt, expiresAt.

5. API Endpoints (Subject to change, fits the project vision so far)

All routes are prefixed with /api and return JSON. Protected routes require a valid JWT, and routes restricted to the post's owner also confirm that the requester created it.

- POST /api/auth/register: create a new user account (public).
- POST /api/auth/login: authenticate and return a JWT (public).

- GET /api/auth/me: return the currently authenticated user (auth required).
- GET /api/items: list nearby items using lat, lng, and radius, with optional status or category filters (public).
- GET /api/items/:id: get a single item by id (public).
- POST /api/items: create a new post (auth required).
- PATCH /api/items/:id: edit an existing post (owner only).
- DELETE /api/items/:id: delete a post (owner only).
- POST /api/items/:id/reserve: reserve an available item (auth required).
- DELETE /api/items/:id/reserve: cancel a reservation (auth required).
- POST /api/items/:id/claim: mark an item as picked up (auth required).
- GET /api/users/:id/items: list all posts by a user (public).

6. Edge Cases and Considerations

- Reservation races: if two users reserve the same item at once, an atomic findOneAndUpdate that only matches an available item ensures exactly one reservation succeeds.
- Expiration: a MongoDB TTL index removes posts after expiresAt, and a scheduled background job releases reservations whose reservedUntil time has passed, returning those items to available.
- Geolocation denied: if the browser blocks location access, the app falls back to manual location entry and a default map center instead of failing.
- Image upload failures and oversized files: the browser enforces limits on file size and type and compresses images before upload, and shows a clear error if Cloudinary rejects the file.
- Authentication errors: expired or missing tokens return 401, and the client clears stored credentials and redirects to login.
- Guarding against invalid actions: users cannot reserve their own posts, and deleting a reserved item updates its status for the user who reserved it.
- Empty and stale states: clear messaging when no items are nearby, and a refresh of results when the map is panned or zoomed.
- Input validation: every write is validated on the server to reject missing or malformed fields.

7. Development Plan

Week 1 (June 24 to June 30): Foundation

Goals: stand up the backend, database, authentication, and the frontend shell, and render posted items on the map and feed.

Tasks - To be distributed evenly amongst the group

- Repository setup, Docker configuration, and shared project scaffolding.
- MongoDB models, the database connection, and the geospatial index.
- Authentication: register, login, JWT, and password hashing.
- Item CRUD endpoints and the query for nearby items.
- React, Tailwind, and routing setup with login and registration pages.
- The Leaflet map, the item feed, and item cards.
- A form for creating a new post, wired to the API.

Deliverables: users can register, log in, post an item with a photo, and see nearby items on a map and feed, and the backend runs locally in Docker.

Week 2 (July 1 to July 7): Features, Polish, and Deployment

Goals: add reservations and automatic expiration, finish image hosting, make the UI responsive, deploy, and prepare the presentation.

Tasks - To be distributed evenly amongst the group

- Reserve, cancel, and claim endpoints with atomic updates.
- TTL expiration and a scheduled job that releases expired reservations.
- Cloudinary image upload integration.
- Reserve and claim flows in the UI, with status badges.
- A responsive and mobile layout pass, with empty and error states.
- Edge case handling and input validation.
- Containerize the client and API, and push both images to Docker Hub.
- Testing, bug fixes, the README, and presentation slides.

Deliverables: a fully working, responsive app with reservations and automatic expiry, with the client and API images published to Docker Hub and the presentation ready for July 7.